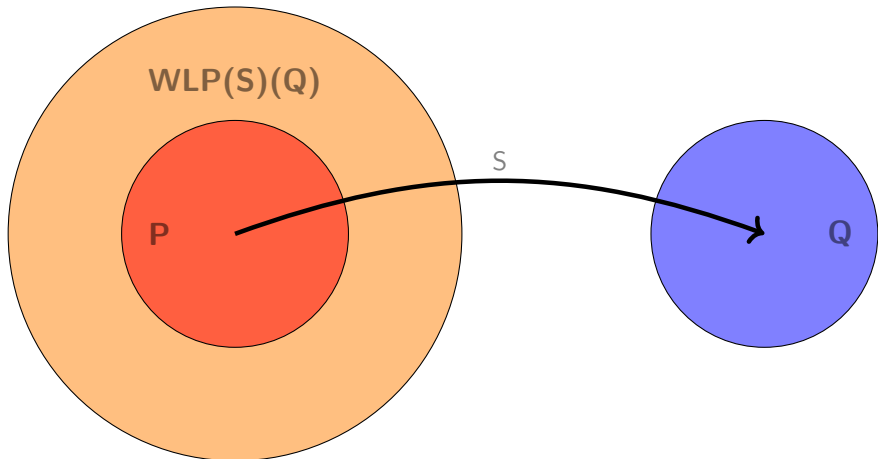
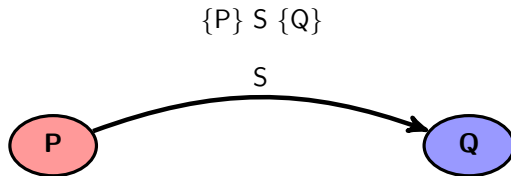
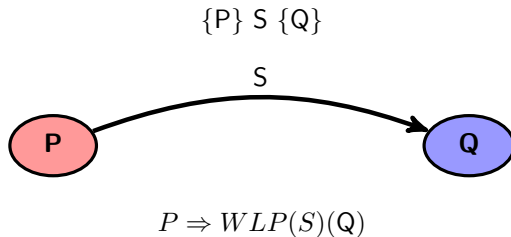
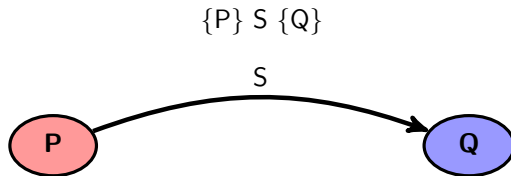


- ① Programs as Predicate Transformers
- ② Generation of Verification Conditions
 - WP calculus in Frama-c
 - First annotation
 - Second annotation
- ③ Annotations
- ④ Contracts
 - Extending C programming language by contracts
 - Playing with variables
 - Ghost Variables









$$P \Rightarrow WLP(S)(Q)$$

Computing $WLP(S)(Q)$?

Listing 1 – an1.c

```
//@ assert l1:  $P(x)$ ;  
  x = e(x);  
//@ assert l2:  $Q(x)$ ;
```

Listing 2 – an1.c

```
//@ assert l1: P(x);  
  x = e(x);  
//@ assert l2: Q(x);
```

► $P(x) \Rightarrow WP(x := e(x))(Q(x))$

Listing 3 – an1.c

```
//@ assert l1: P(x);  
  x = e(x);  
//@ assert l2: Q(x);
```

- ▶ $P(x) \Rightarrow WP(x := e(x))(Q(x))$
- ▶ $P(x) \Rightarrow Q[x \mapsto e(x)]$

Listing 4 – an1.c

```
//@ assert l1: P(x);  
  x = e(x);  
//@ assert l2: Q(x);
```

- ▶ $P(x) \Rightarrow WP(x := e(x))(Q(x))$
- ▶ $P(x) \Rightarrow Q[x \mapsto e(x)]$
- ▶ $P(x1) \Rightarrow Q[x \mapsto e(x1)]$ (renaming of free occurrences of x by $x1$)

Listing 5 – an1.c

```
//@ assert l1: P(x);  
  x = e(x);  
//@ assert l2: Q(x);
```

- ▶ $P(x) \Rightarrow WP(x := e(x))(Q(x))$
- ▶ $P(x) \Rightarrow Q[x \mapsto e(x)]$
- ▶ $P(x1) \Rightarrow Q[x \mapsto e(x1)]$ (renaming of free occurrences of x by $x1$)
- ▶ $P(x1) \wedge x = e(x1) \Rightarrow Q(x)$

Listing 6 – an1.c

```
//@ assert l1: P(x);  
  x = e(x);  
//@ assert l2: Q(x);
```

- ▶ $P(x) \Rightarrow WP(x := e(x))(Q(x))$
- ▶ $P(x) \Rightarrow Q[x \mapsto e(x)]$
- ▶ $P(x1) \Rightarrow Q[x \mapsto e(x1)]$ (renaming of free occurrences of x by $x1$)
- ▶ $P(x1) \wedge x = e(x1) \Rightarrow Q(x)$
- ▶ $P(x1) \wedge x = e(x1) \Rightarrow Q(x)$

Listing 7 – an1.c

```
//@ assert l1: P(x);  
  x = e(x);  
//@ assert l2: Q(x);
```

- ▶ $P(x) \Rightarrow WP(x := e(x))(Q(x))$
- ▶ $P(x) \Rightarrow Q[x \mapsto e(x)]$
- ▶ $P(x1) \Rightarrow Q[x \mapsto e(x1)]$ (renaming of free occurrences of x by $x1$)
- ▶ $P(x1) \wedge x = e(x1) \Rightarrow Q(x)$
- ▶ $P(x1) \wedge x = e(x1) \Rightarrow Q(x)$
- ▶ $\vdash P(x1) \wedge x = e(x1) \Rightarrow Q(x)$

Listing 10 – an1.c

```
int main(void) {
    signed long int x,y,z; //      int x,y,z;
    x = 1;
    /*@ assert x == 1; */
    y = 2;
    /*@ assert x == 1 && y == 2; */
    z = x * y;
    /*@ assert x == 1 && y == 1 && z == 2; */
    return 0;
}
```

```
[kernel] Parsing an1.c (with preprocessing)
[wp] Running WP plugin...
[wp] Warning: Missing RTE guards
[wp] 2 goals scheduled
[wp] Proved goals:      4 / 4
    Terminating:      1
    Unreachable:       1
    Qed:                2
```

Assume {

```

Type: is_sint32(x) /\ is_sint32(y).
(* Initializer *)
Init: x = 12.
(* Initializer *)
Init: y = 24.
}

```

Prove: $(2 * x) = y$.

Prover Qed returns Valid

Annotation simple (I)

Goal Assertion 'l2' (file an1.c, line 5):

Assume {

Type: $\text{is_sint32}(x) \wedge \text{is_sint32}(x_1) \wedge \text{is_sint32}(y).$

```
(* Initializer *)
```

Init: $x_1 = 12$.

```
(* Initializer *)
```

Init: $y = 24$.

```
(* Assertion 'l1' *)
```

Have: $(2 * x_1) = y.$

Have: $(1 + x_1) = x.$

}

Prove: $(2 + y) = (2 * x)$.

Prover Qed returns Valid

```
[kernel] Parsing an1.c (with preprocessing)
[wp] Running WP plugin...
[wp] Warning: Missing RTE guards
[wp] 2 goals scheduled
[wp] Proved goals:      4 / 4
    Terminating:      1
    Unreachable:       1
    Qed:                2
```

Listing 11 – an2.c

```
void ex(void) {  
    int x=12,y=24;  
    //@ assert l1: 2*x == y;  
    x = x+1;  
    //@ assert l2: y == 2*(x-1);  
    x = x+2;  
    //@ assert l3: y+6 == 2*x;  
}
```

Annotation simple (II)

```
[kernel] Parsing an2.c (with preprocessing)
```

```
[wp] Running WP plugin...
```

```
[wp] Warning: Missing RTE guards
```

[wp] 3 goals scheduled

```
[wp] Proved goals:      5 / 5
```

Terminating: 1

```
Unreachable:      1
```

Qed: 3

Annotation simple (ii)

Goal Assertion 'l3' (file an2.c, line 7):

Assume {

```
Type: is_sint32(x) /\ is_sint32(x_1) /\ is_sint32(x_2) /\ is_s
```

(* Initializer *)

Init: $x_2 = 12$.

```
(* Initializer *)
```

Init: $y = 24$.

```
(* Assertion 'l1' *)
```

Have: $(2 * x_2) = y.$

Have: $(1 + x_2) = x_1$.

```
(* Assertion 'l2' *)
```

Have: $(2 + y) = (2 * x_1).$

Have: $(2 + x_1) = x.$

}

Prove: $(6 + y) = (2 * x)$.

Prover Qed returns Valid

Listing 12 – an2bis.c

```
void ex(void) {  
    int x=12,y=24;  
    //@ assert l1:  $2*x == y$ ;  
    x = x+1;  
    //@ assert l2:  $y == 2*(x-1)$ ;  
    x = x+2;  
    //@ assert l3:  $y+6 == 2*x$ ;  
}
```

Writing a simple contract

variables x

requires $x \geq 0 \wedge x \leq 10$;

ensures $\begin{cases} x \% 2 = 0 \Rightarrow 2 \cdot \text{result} = x +; \\ x \% 2 \neq 0 \Rightarrow 2 \cdot \text{result} = x - 1; \end{cases}$

begin

int y ;

$y = x/2$;

return(y);

end

▶ result is the value returned by the command *return*(y).

▶ *return*(y) is equivalent to $\text{result} := y$.

(Writing a simple contract.)

Listing 13 – project-divers/annotation.c

```
/*@ requires x >= 0 && x <= 10;
   @ assigns \nothing;
   @ ensures x % 2 == 0 ==> 2*\result == x;
   @ ensures x % 2 != 0 ==> 2*\result == x-1;
*/
int f(int x)
{
  int y;
  y = x/2 ;
  return(y);
}
```

(Writing a simple contract.)

Listing 14 – project-divers/annotationwp.c

```
/*@ requires 0 <= x && x <= 10;
   @ assigns \nothing;
   @ ensures x % 2 == 0 ==> 2*\result == x;
   @ ensures x % 2 != 0 ==> 2*\result == x-1;
   @*/

int f(int x)
{
  /*@ assert x % 2 == 0 ==> 2* (x / 2) == x; */
  /*@ assert x % 2 != 0 ==> 2* (x / 2) == x-1; */
  int y;
  /*@ assert x % 2 == 0 ==> 2* (x / 2) == x; */
  /*@ assert x % 2 != 0 ==> 2* (x / 2) == x-1; */
  y = x / 2;
  /*@ assert x % 2 == 0 ==> 2*y == x; */
  /*@ assert x % 2 != 0 ==> 2*y == x-1; */
  return(y);
  /*@ assert x % 2 == 0 ==> 2*y == x; */
  /*@ assert x % 2 != 0 ==> 2*y == x-1; */
}
```

Property to check

$$x \geq 0 \wedge x \leq 10 \Rightarrow \begin{cases} x \% 2 \neq 0 \Rightarrow 2 \cdot (x/2) = x-1 \\ x \% 2 = 0 \Rightarrow 2 \cdot (x/2) = x \end{cases}$$

(Checking the precondition.)

Listing 15 – project-divers/annotation0.c

```
/*@ requires x >= 0 && x < 0;  
  @ assigns \nothing;  
  @ ensures \result == 0;  
  @*/  
int annotation0(int x)  
{  
  int y;  
  y = y / (x-x);  
  return(y);  
}
```

(Checking the precondition.)

Listing 16 – `project-divers/annotation0wp.c`

```

/*@ requires x >= 0 && x < 0;
   @ assigns \nothing;
   @ ensures \result == 0;
   @*/
int annotation(int x)
{
    /*@ assert y / (x-x) == 0; */
    int y;
    /*@ assert y / (x-x) == 0; */
    y = y / (x-x);
    /*@ assert y == 0; */
    return(y);
    /*@ assert y == 0; */
}

```

Property to check

$$x \geq 0 \wedge < 0 \Rightarrow y/(x-x) = 0$$

Transformations of annotated programs (1)

```
//@ assert  $P(v0, v) :$ 
 $S1; S2$ 
//@ assert  $Q(v0, v) :$ 
```

- ▶ Applying the property :

$$wp(S1; S2)(A) = wp(S1)(wp(S2)(A))$$

```
//@ assert  $P(v0, v)$  :
S1;
//@ assert  $wp(S2)(Q(v0, v))$  :
S2;
//@ assert  $Q(v0, v)$  :
```

```
//@ assert  $P(v0, v)$  :
//@ assert  $xp(S1)(wp(S2)(Q(v0, v)))$  :
 $S1$ ;
//@ assert  $wp(S2)(Q(v0, v))$  :
 $S2$ ;
//@ assert  $Q(v0, v)$  :
```

Transformations of annotated programs (2)

```

//@ assert  $P(v0, v)$  :
IF  $B$  THEN
     $S1$ 
ELSE
     $S2$ 
FI
//@ assert  $Q(v0, v)$  :

```

► Applying the property :

$$wp(if(B, S1, S2))(A) = b \wedge wp(S1)(A) \vee \neg B \wedge wp(S2)(A).$$

```

//@ assert  $P(v_0, v)$  :
IF  $B$  THEN
     $S_1$ 
ELSE
     $S_2$ 
FI
//@ assert  $Q(v_0, v)$  :

```

```

//@ assert  $P(v0, v)$  :
IF  $B$  THEN
     $S1$ 
//@ assert  $Q(v0, v)$  :
ELSE
     $S2$ 
//@ assert  $Q(v0, v)$  :
FI
//@ assert  $Q(v0, v)$  :

```

Transformations of annotated programs (2)

```
//@ assert  $P(v0, v)$  :  
IF  $B$  THEN  
   $S1$   
//@ assert  $Q(v0, v)$  :  
ELSE  
   $S2$   
//@ assert  $Q(v0, v)$  :  
FI  
//@ assert  $Q(v0, v)$  :
```

Transformations of annotated programs (2)

```

//@ assert  $P(v0, v)$  :
IF  $B$  THEN
     $S1$ 
//@ assert  $Q(v0, v)$  :
ELSE
     $S2$ 
//@ assert  $Q(v0, v)$  :
FI
//@ assert  $Q(v0, v)$  :

```

```

//@ assert  $P(v0, v)$  :
IF  $B$  THEN
//@ assert  $B \wedge wp(S2)(Q(v0, v))$  :
     $S1$ 
//@ assert  $Q(v0, v)$  :
ELSE
//@ assert  $\neg B \wedge wp(S2)(Q(v0, v))$  :
     $S2$ 
//@ assert  $Q(v0, v)$  :

```

Transformations of annotated programs (2)

```
//@ assert  $P(v0, v)$  :  
IF  $B$  THEN  
   $S1$   
  //@ assert  $Q(v0, v)$  :  
ELSE  
   $S2$   
  //@ assert  $Q(v0, v)$  :  
FI  
//@ assert  $Q(v0, v)$  :
```

```
//@ assert  $P(v0, v)$  :  
IF  $B$  THEN  
  //@ assert  $B \wedge wp(S2)(Q(v0, v))$  :  
   $S1$   
  //@ assert  $Q(v0, v)$  :  
ELSE  
  //@ assert  $\neg B \wedge wp(S2)(Q(v0, v))$  :  
   $S2$   
  //@ assert  $Q(v0, v)$  :
```

```
//@ assert  $P(v0, v)$  :  
IF  $B$  THEN  
  //@ assert  $b \wedge wp(S1)(Q(v0, v))$  :  
   $S1$   
  //@ assert  $Q(v0, v)$  :  
ELSE  
  //@ assert  $\neg b \wedge wp(S2)(Q(v0, v))$  :  
   $S2$   
  //@ assert  $Q(v0, v)$  :  
FI  
//@ assert  $Q(v0, v)$  :
```

Transformations of annotated programs (2)

```

//@ assert  $P(v_0, v)$  :
IF  $B$  THEN
     $S_1$ 
//@ assert  $Q(v_0, v)$  :
ELSE
     $S_2$ 
//@ assert  $Q(v_0, v)$  :
FI
//@ assert  $Q(v_0, v)$  :

```

```

//@ assert  $P(v0, v)$  :
IF  $B$  THEN
//@ assert  $B \wedge wp(S2)(Q(v0, v))$  :
     $S1$ 
//@ assert  $Q(v0, v)$  :
ELSE
//@ assert  $\neg B \wedge wp(S2)(Q(v0, v))$  :
     $S2$ 
//@ assert  $Q(v0, v)$  :

```

```

//@ assert  $P(v0, v)$  :
IF  $B$  THEN
//@ assert  $b \wedge wp(S1)(Q(v0, v))$  :
     $S1$ 
//@ assert  $Q(v0, v)$  :
ELSE
//@ assert  $\neg b \wedge wp(S2)(Q(v0, v))$  :
     $S2$ 
//@ assert  $Q(v0, v)$  :
FI
//@ assert  $Q(v0, v)$  :

```

- ▶ $b \wedge P(v0, v) \Rightarrow$
 $b \wedge wp(S1)(Q(v0, v))$
- ▶ $\neg b \wedge P(v0, v) \Rightarrow$
 $\neg b \wedge wp(S2)(Q(v0, v))$

```
//@ assert  $P(v0, v)$  :  
//@ loop invariant  $I(v0, v)$  :  
WHILE  $B$  THEN  
   $S$   
OD  
//@ assert  $Q(v0, v)$  :
```

- ▶ Applying the iteration rule of Hoare Logic :

Transformations of annotated programs (3)

```
//@ assert  $P(v_0, v)$  :
//@ loop invariant  $I(v_0, v)$  :
WHILE  $B$  THEN
   $S$ 
OD
//@ assert  $Q(v_0, v)$  :
```

```

//@ assert  $P(v0, v)$  :
//@ loop invariant  $I(v0, v)$  :
//@ assert  $I(v0, v)$  :
WHILE  $B$  THEN
  //@ assert  $b \wedge I(v0, v)$  :
   $S$ 
  //@ assert  $I(v0, v)$  :
OD
//@ assert  $Q(v0, v)$  :

```

- ▶ Applying the iteration rule of Hoare Logic :

Transformations of annotated programs (3)

```
//@ assert  $P(v_0, v)$  :
//@ loop invariant  $I(v_0, v)$  :
WHILE  $B$  THEN
   $S$ 
OD
//@ assert  $Q(v_0, v)$  :
```

- ▶ Applying the iteration rule of Hoare Logic :

```

//@ assert  $P(v0, v)$  :
//@ loop invariant  $I(v0, v)$  :
//@ assert  $I(v0, v)$  :
WHILE  $B$  THEN
  //@ assert  $b \wedge I(v0, v)$  :
     $S$ 
  //@ assert  $I(v0, v)$  :
OD
//@ assert  $Q(v0, v)$  :

```

- ▶ $b \wedge I(v0, v) \Rightarrow wp(S)(I(v0, v))$
- ▶ $P(v0, v) \Rightarrow I(v0, v)$
- ▶ $\neg b \wedge I(v0, v) \Rightarrow Q(v0, v)$

- (8 octobre 2026) 26/48

- ▶ Assertions at a control point of the program

```
/*@ assert pred; */
```

```
//@ assert pred;
```

- ▶ Assertions at a control point of the program components.

```
/*@ for id1,id2, ..., idn: assert pred; */
```

(Incrementing a number)

Listing 17 – project-divers/compwp0.c

```
#define x0 5
/*@ assigns \nothing; */
int exemple() {
    int x=x0;
    //@ assert x == x0;
    x = x + 1;
    //@ assert x == x0+1;
    return x;
}
```

(Incrementing a number)

Listing 18 – project-divers/compwp0wp.c

```
#define x0 5
/*@ assigns \nothing; */
int exemple() {
    //@ assert x0 == x0;
    //@ assert x0+1 == x0+1;
    int x=x0;
    //@ assert x == x0;
    //@ assert x+1 == x0+1;
    x = x + 1;
    //@ assert x == x0+1;
    return x;
}
```

Summary on annotations and assertions

- ▶ requires
- ▶ assigns
- ▶ ensures
- ▶ decreases
- ▶ predicate
- ▶ logic
- ▶ lemma

- ▶ The calling function should guarantee the required condition or precondition introduced by the clause requires $P1 \wedge \dots \wedge Pn$ at the calling point.
- ▶ The called function returns results that are ensured by the clause ensures $E1 \wedge \dots \wedge Em$; ensures clause expresses a relationship between the initial values of variables and the final values.
- ▶ initial values of a variable v is denoted $\backslash old(v)$
- ▶ The variables which are not in the set $L1 \cup \dots \cup Lp$ are not modified.

Listing 19 – contrat

```
/*@ requires P1;...;requires Pn;
   @ assigns L1;...;assigns Lm;
   @ ensures E1;...;ensures Ep;
   @*/
```

(Division)

Listing 20 – project-divers/annotation.c

```
/*@ requires x >= 0 && x <= 10;  
  @ assigns \nothing;  
  @ ensures x % 2 == 0 ==> 2*\result == x;  
  @ ensures x % 2 != 0 ==> 2*\result == x-1;  
*/  
int f(int x)  
{  
  int y;  
  y = x/2 ;  
  return(y);  
}
```

(Division)

Listing 21 – `project-divers/annotationwp.c`

```

/*@ requires 0 <= x && x <= 10;
   @ assigns \nothing;
   @ ensures  x % 2 == 0 ==> 2*\result == x;
   @ ensures  x % 2 != 0 ==> 2*\result == x-1;
   */

int f(int x)
{
  /*@ assert x % 2 == 0 ==> 2* (x / 2) == x; */
  /*@ assert x % 2 != 0 ==> 2* (x / 2) == x-1; */
  int y;
  /*@ assert x % 2 == 0 ==> 2* (x / 2) == x; */
  /*@ assert x % 2 != 0 ==> 2* (x / 2) == x-1; */
  y = x / 2;
  /*@ assert x % 2 == 0 ==> 2*y == x; */
  /*@ assert x % 2 != 0 ==> 2*y == x-1; */
  return(y);
  /*@ assert x % 2 == 0 ==> 2*y == x; */
  /*@ assert x % 2 != 0 ==> 2*y == x-1; */
}

```

Property to check

$$x \geq 0 \wedge x < 0; \Rightarrow \left(\begin{array}{l} x \% 2 = 0 \Rightarrow 2 \cdot (x/2) = x \\ x \% 2 \neq 0 \Rightarrow 2 \cdot (x/2) = x-1 \end{array} \right)$$

Listing 22 – project-divers/annotation0.c

(8 octobre 2026) 32/48

(Precondition)

Listing 23 – project-divers/annotation0wp.c

```
/*@ requires x >= 0 && x < 0;  
  @ assigns \nothing;  
  @ ensures \result == 0;  
  @*/  
int annotation(int x)  
{  
  /*@ assert y / (x-x) == 0; */  
  int y;  
  /*@ assert y / (x-x) == 0; */  
  y = y / (x-x);  
  /*@ assert y == 0; */  
  return(y);  
  /*@ assert y == 0; */  
}
```

Property to check

$$0 \leq x \wedge x \leq 10 \Rightarrow y/(x-x) = 0$$

Definition of a contract (programming)

- ▶ Define the program codefact for computing mathfact (How to compute?)
- ▶ Define the algorithm computing the function mathfact

(factorial how)

Listing 25 – project-factorial/factorial.c

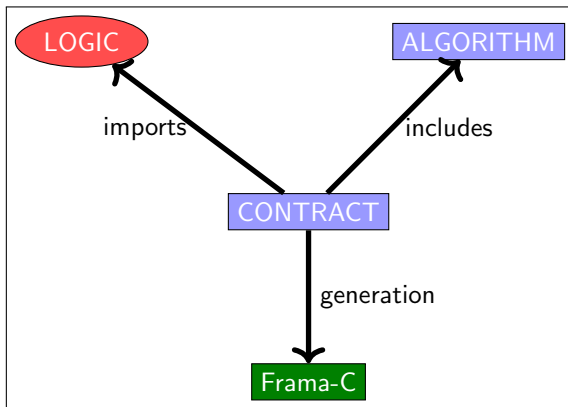
```
#include "factorial.h"

int codelfact(int n) {
    int y = 1;
    int x = n;
    /*@ loop invariant x >= 1 && x <= n && mathfact(n) == y * mathfact(x);
       loop assigns x, y;
       loop variant x;
    */
    while (x != 1) {
        y = y * x;
        x = x - 1;
    };
    return y;
}
```

Definition of a contract (approach)

- ▶ The specification of a function (`mathfact`) to compute requires to define it mathematically.
- ▶ The definition is stated in an axiomatic framework and is preferably inductive (`mathfact`) which is used in assertions or theorems or lemmas.
- ▶ The relationship between the computed value (`\result`) and the mathematical value (`mathfact(n)`) is stated in the ensures clause :

$$\text{\result} == \text{mathfact}(n)$$
- ▶ The main property to prove is `codefact(n) == mathfact(n)` : Calling `codefact` for `n` returns a value equal to `mathfact(n)`.



Listing 26 – contrat

```

/*@ requires P;
   @ behavior b1:
       @ assumes A1;
       @ requires R1 ;
       @ assigns L1;
       @ ensures E1;
   @ behavior b2:
       @ assumes A2;
       @ requires R2;
       @ assigns L2;
       @ ensures E2;
   @*/

```

(Pairs of integers)

Listing 27 – project-divers/structures.h

```
#ifndef _STRUCTURE_H
```

```
struct s {
    int q;
    int r;
};
```

```
#endif
```

Division should not return silly expressions !

(Specification)

Listing 28 – project-divers/division.h

```
#ifndef _A_H
#define _A_H
#include "structures.h"
/*@ requires a >= 0 && b >= 0;
    @ requires b != 0;
    @ ensures 0 <= \result.r;
    @ ensures \result.r < b;
    @ ensures a == b * \result.q + \result.r;
*/
struct s division(int a, int b);
#endif
```

(Algorithm)

Listing 29 – project-divers/division.c

```
#include <stdio.h>
#include <stdlib.h>
#include "division.h"

struct s division(int a, int b)
{
    int rr = a;
    int qq = 0;
    struct s silly = {-1,-1};
    struct s resu;
    if (b == 0) {
        return silly;
    }
    else
    {
        /*@
        loop invariant
        ( a == b*qq + rr) &&
        rr >= 0;
        loop assigns rr,qq;
        loop variant rr;
        */
        while (rr >= b) { rr = rr - b; qq=qq+1;};
        resu.q = qq;
        resu.r = rr;
        return resu;
    }
}
```


(Invariant de boucle)

Listing 31 – project-divers/anno6.c

```

/*@ requires a >= 0 && b >= 0;
   ensures 0 <= \result;
   ensures \result < b;
   ensures \exists integer k; a == k * b + \result;
*/
int rem(int a, int b) {
    int r = a;
    /*@
       loop invariant
       (\exists integer i; a == i * b + r) &&
       r >= 0;
       loop assigns r;
    */
    while (r >= b) { r = r - b; };
    return r;
}

```

- ▶ $\backslash old(x)$ is the value of the variable when the function is called.
- ▶ It can be used in the postcondition of the *ensures* clause.

(Modifying variables while calling)

Listing 32 – project-divers/old1.c

```

/*@ requires \valid(a) && \valid(b);
   @ assigns *a,*b;
   @ ensures  *a == \at(*a,Pre) +2;
   @ ensures  *b == \at(*b,Pre)+\at(*a,Pre)+2;

           @ ensures  \result == 0;

*/
int old(int *a, int *b) {
    int x,y;
    x = *a;
    y = *b;
    x=x+2;
    y = y +x;

    *a = x;
    *b = y;
    return 0 ;
}

```

- ▶ $\backslash at(e, id)$ is the value of e at the control point id .
- ▶ id should occur before $\backslash at(e, id)$
- ▶ id is one of the possible expressions : Pre, Here, Old, Post, LoopEntry, LoopCurrent, Init
- ▶ $\backslash old(e)$ is equivalent to $\backslash at(e, Old)$

(label Pre)

Listing 33 – project-divers/at1.c

```

/*@
  requires  \valid(a) && \valid(b);
  assigns  *a,*b;
  ensures   *a == \old(*a)+2;
  ensures   *b == \old(*b)+\old(*a)+2;
*/
int at1(int *a, int *b) {
  //@ assert *a == \at(*a, Pre);
  *a = *a + 1;
  //@ assert *a == \at(*a, Pre)+1;
  *a = *a + 1;
  //@ assert *a == \at(*a, Pre)+2;
  *b = *b + *a;
  //@ assert *a == \at(*a, Pre)+2 && *b == \at(*b, Pre)+\at(*a, Pre)+2;
  return 0;
}

```


(autre label)

Listing 34 – project-divers/at2.c

```
void f (int n) {
  for (int i = 0; i < n; i++) {
    /*@ assert \at(i, LoopEntry) == 0; */
    int j=0;
    while (j++ < i) {
      /*@ assert \at(j, LoopEntry) == 0; */
      /*@ assert \at(j, LoopCurrent) + 1 == j; */
    }
  }
}
```

(otherlabel)

Listing 35 – project-divers/change1.c

```
/*@ requires \valid(a) && *a >= 0;
   @ assigns *a;
   @ ensures *a == \old(*a)+2 && \result == 0;
*/
int change1(int *a)
{
    int x = *a;
    x = x + 2;
    *a = x;
    return 0;
}
```

- ▶ A variable called *ghost* allows to model a computed value useful for stating a model property : the ghost variable is hidden for the computer but not for the model.
- ▶ It should not change the semantics of others variables and should not change the effective variables.

(Bug)

Listing 36 – project-divers/ghost2.c

```
int f (int x, int y) {
    //@ghost int z=x+y;
    switch (x) {
    case 0: return y;
    //@ ghost case 1: z=y;
    // above statement is correct.
    //@ ghost case 2: { z++; break; }
    // invalid , would bypass the non-ghost default
    default: y++; }
    return y; }

int g(int x) { //@ ghost int z=x;
    if (x>0){return x;}
    //@ ghost else { z++; return x; }
    // invalid , would bypass the non-ghost return
    return x+1; }
```

(Ghost variable)

Listing 37 – project-divers/ghost1.c

```

/*@ requires a >= 0 && b >= 0;
   ensures 0 <= \result;
   ensures \result < b;
   ensures \exists integer k; a == k * b + \result; */
int rem(int a, int b) {
    int r = a;
    /*@ ghost    int q=0;    */
    /*@
       loop invariant
       a == q * b + r &&
       r >= 0 && r <= a;
       loop assigns r;
       loop assigns q;
    // loop variant r;
    */
    while (r >= b) {
        r = r - b;
    /*@ ghost    q = q+1;    */
    };
    return r;
}

```